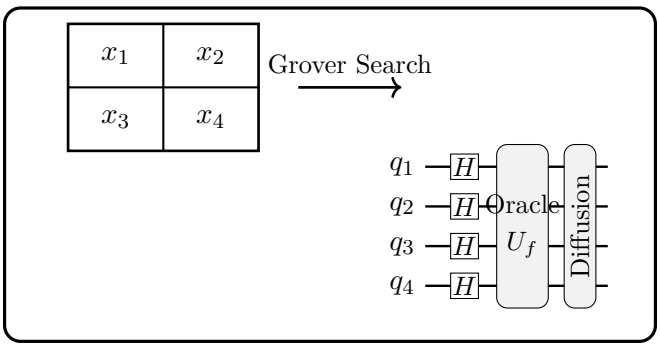
A Quantum Computing Project

Solving a 2×2 Sudoku with Grover Search

With and Without Simple Error Correction



Taha Malik

April 8, 2026

Contents

1 Introduction **2**

2 The 2×2 Sudoku Problem **2**

3 Encoding the Puzzle into Qubits **3**

4 From Sudoku Rules to Boolean Logic **3**

5 Part I: Ideal Circuit Without Error Correction **3**
5.1 Registers used . . . 3
5.2 Step 1: Prepare the superposition . . . 4
5.3 Why the phase ancilla is prepared in $|-\rangle$ . . . 4
5.4 Step 2: The oracle . . . 4

6 Circuit Implementation **4**
6.1 The Oracle operator . . . 4
6.2 Step 3: The Diffusion operator . . . 5
6.3 Expected ideal output . . . 6

7 Part II: Adding Simple Error Correction **6**
7.1 Why error correction . . . 6
7.2 The Three-Qubit Repetition Code . . . 6
7.3 Syndrome extraction . . . 6

8 Error-Corrected Sudoku Circuit **7**

9 Expected Results **7**

10 From Repetition Codes to Surface Codes **8**
10.1 Repetition code as a 1D chain . . . 8
10.2 Stabilizer formalism . . . 8
10.3 Extending to two dimensions . . . 8
10.4 Errors as geometric objects . . . 9
10.5 Key insight . . . 10

11 Conclusion **10**

A Qiskit Sketch for the Ideal Circuit **11**

B Possible Extensions **12**

Abstract

This report present a compact implementation of Grover's algorithm [1]. for a 2×2 Sudoku. The puzzle is encoded into qubits, its constraints are implemented as a reversible phase oracle using ancillas and phase kickback, and Grover diffusion amplifies the valid assignment. The discussion begins with the ideal circuit model, emphasizing reversible logic, ancilla management, and uncomputation, and then extends to a three-qubit repetition code protecting the data register against single bit-flip errors. Finally, two-dimensional surface codes are briefly introduced.

1 Introduction

Grover's algorithm considers a search space of size N together with a Boolean function $f(x) \in \{0, 1\}$ that identifies valid solutions. Classically, finding such a solution may require checking on the order of N candidates. Grover's algorithm reduces this to on the order of $\sqrt{N}$ oracle queries.

The procedure begins by preparing an equal superposition over all computational basis states $|x\rangle$. An oracle then marks valid solutions by applying a phase flip,

$$U_f |x\rangle = (-1)^{F(x)} |x\rangle . \tag{1.1}$$

so that correct states acquire a negative sign while others remain unchanged. This allows the diffusion operator

$$D = 2 |s\rangle \langle s| - I, \quad |s\rangle = \frac{1}{\sqrt{N}} \sum_x |x\rangle , \tag{1.2}$$

to amplify the amplitude of marked states. Repeating these two steps rotates the system toward the solution, allowing it to be measured with high probability.

We develop a complete implementation of this approach for a 2×2 Sudoku.

2 The 2×2 Sudoku Problem

Consider the 2×2 grid

$$\begin{matrix} x_1 & x_2 \\ x_3 & x_4 \end{matrix}$$

where each variable takes values in $1, 2$. The Sudoku constraints require that entries differ along rows and columns:

$$x_1 \neq x_2, \quad x_3 \neq x_4, \quad x_1 \neq x_3, \quad x_2 \neq x_4. \tag{2.1}$$

Imposing the clue $x_1 = 1$ yields a unique valid assignment,

$$\boxed{\begin{matrix} 1 & 2 \\ 2 & 1 \end{matrix}}.$$

Thus, the problem reduces to identifying a unique valid solution.

3 Encoding the Puzzle into Qubits

Because each cell can take only two values, one qubit per cell is sufficient. We use the convention

$$|0\rangle \leftrightarrow 1, \qquad |1\rangle \leftrightarrow 2$$

so the four Sudoku variables are encoded into the register as $|q_1 q_2 q_3 q_4\rangle$ and the valid Sudoku solution is represented by $|0110\rangle$. This turns the original puzzle into a search problem over the 16 computational basis states of four qubits.

4 From Sudoku Rules to Boolean Logic

To build the oracle, we first translate the Sudoku constraints and clues into Boolean conditions on the qubits. The puzzle is valid when the following are true:

$$q_1 = 0, \qquad q_1 \oplus q_2 = 1, \qquad q_3 \oplus q_4 = 1, \qquad q_1 \oplus q_3 = 1, \qquad q_2 \oplus q_4 = 1. \tag{4.1}$$

Equivalently, define

$$F(q_1, q_2, q_3, q_4) = (\neg q_1) \wedge (q_1 \oplus q_2) \wedge (q_3 \oplus q_4) \wedge (q_1 \oplus q_3) \wedge (q_2 \oplus q_4). \tag{4.2}$$

If we denote a candidate assignment by $x = (q_1, q_2, q_3, q_4) \in \{0,1\}^4$, then $F(x) = 1$ if and only if x is the valid Sudoku solution.

5 Part I: Ideal Circuit Without Error Correction

5.1 Registers used

The ideal circuit uses three types of qubits:

Register	Number of qubits	Purpose
Data qubits	4	store the Sudoku assignment
Constraint ancillas	5	store intermediate Boolean checks
Phase ancilla	1	implements phase kickback

Thus we begin with the register

$$|0000\rangle_{data} |00000\rangle_{constraint} |-\rangle_{phase}, \qquad |-\rangle_{phase} = \frac{|0\rangle - |1\rangle}{\sqrt{2}}.$$ [1]

[1] Equivalently, $|-\rangle = H|1\rangle$, so it can be prepared from $|0\rangle$ by applying X followed by H.

5.2 Step 1: Prepare the superposition

Applying Hadamard gates to the four data qubits gives

$$H^{\otimes 4} |0000\rangle = \frac{1}{4} \sum_{x \in \{0,1\}^4} |x\rangle \,, \tag{5.1}$$

which is the equal superposition over all 16 candidate Sudoku assignments.

5.3 Why the phase ancilla is prepared in $|-\rangle$

The special role of $|-\rangle$ comes from the identity

$$X |-\rangle = - |-\rangle \,, \tag{5.2}$$

where X is the Pauli-X (bit-flip) gate. Therefore, if a multi-controlled CNOT (i.e., a multi-controlled X) flips the phase ancilla, the net effect is to multiply the corresponding branch of the state by -1. This mechanism converts a Boolean computation into a phase oracle.

5.4 Step 2: The oracle

The oracle acts on computational basis states as $U_f |x\rangle = (-1)^{F(x)} |x\rangle$. To achieve this, we compute the individual checks into ancillas:

$$a_1 = \neg q_1, \quad a_2 = q_1 \oplus q_2, \quad a_3 = q_3 \oplus q_4, \quad a_4 = q_1 \oplus q_3, \quad a_5 = q_2 \oplus q_4. \tag{5.3}$$

If all five ancillas are 1, a multi-controlled X gate flips the phase ancilla, marking the valid Sudoku state. The ancillas are subsequently uncomputed and returned to the state $|00000\rangle$.

6 Circuit Implementation

6.1 The Oracle operator

Figure 1 shows the circuit implementation of the oracle U_F.

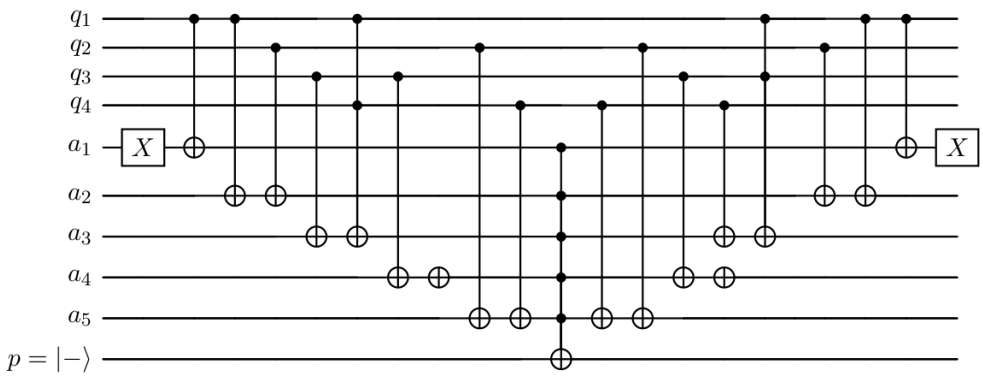


Figure 1: Circuit implementation of the oracle U_F. The circuit first computes the five constraint checks into ancilla qubits, then uses them to control a phase flip through the phase ancilla, and finally uncomputes the workspace.

6.2 Step 3: The Diffusion operator

After the oracle marks the valid state, the diffusion operator (Eq. (1.2)) is applied. A standard implementation is

$$D = H^{\otimes 4} X^{\otimes 4} \left(2 \left|1111\right\rangle \left\langle 1111\right| - I\right) X^{\otimes 4} H^{\otimes 4}. \tag{6.1}$$

Figure 2 shows one Grover iteration, with the oracle U_F treated as a black box and the diffusion step implemented explicitly. See Figure 1 for a detailed implementation of U_F

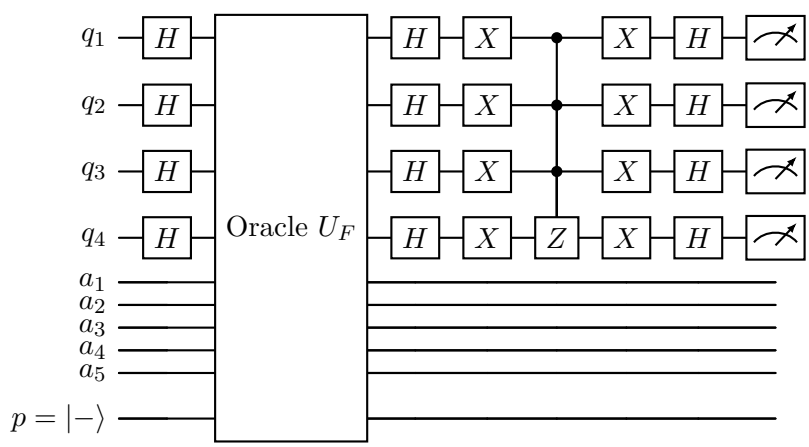


Figure 2: One Grover iteration with the diffusion operator written explicitly on the data qubits. The ancilla qubits are used only inside the oracle and are untouched by the diffusion step.

6.3 Expected ideal output

After roughly $r \approx \left\lfloor \frac{\pi}{4}\sqrt{\frac{N}{M}} \right\rfloor = \left\lfloor \frac{\pi}{4}\sqrt{16} \right\rfloor = 3$ Grover iterations, the data register should be measured as $|0110\rangle$ with high probability.

7 Part II: Adding Simple Error Correction

7.1 Why error correction

Real quantum devices are noisy: bit flips, phase flips, decoherence, and gate imperfections degrade the coherent phases required by Grover's algorithm, reducing success probability. We focus on a simplified case:

$$\boxed{\text{protect against isolated single bit-flip errors on the data qubits.}}$$

7.2 The Three-Qubit Repetition Code

The repetition code encodes one logical qubit into three physical qubits:

$$|0_L\rangle = |000\rangle\,, \qquad |1_L\rangle = |111\rangle\,.$$

Thus a general qubit $\alpha\,|0\rangle + \beta\,|1\rangle$ is encoded as $\alpha\,|000\rangle + \beta\,|111\rangle$. If one physical qubit flips, the other two still preserve the original value. For example,

$$|000\rangle \to |100\rangle$$

corresponds to logical 0 by majority vote, and

$$|111\rangle \to |101\rangle$$

corresponds to logical 1. The code therefore corrects a single bit-flip error within the block.

The encoding is implemented by taking one data qubit and two ancillas in $|0\rangle$ and applying two CNOT gates, producing the three-qubit block.

7.3 Syndrome extraction

To detect which qubit flipped, one measures the parity checks

$$Z_1 Z_2, \qquad Z_2 Z_3, \tag{7.1}$$

where Z_i denotes the Pauli-Z operator acting on the i-th physical qubit (with identity on all others). These checks compare whether neighboring qubits agree. Because valid code states are eigenstates of these operators, measuring them does not collapse the logical state; it only reveals the error syndrome. These measurements extract only parity information (whether qubits agree or disagree), not the encoded logical value.

The syndrome identifies the error location:

Syndrome (Z_1Z_2, Z_2Z_3)	Error location	Correction
$(+1, +1)$	none	do nothing
$(-1, +1)$	qubit 1	X_1
$(-1, -1)$	qubit 2	X_2
$(+1, -1)$	qubit 3	X_3

8 Error-Corrected Sudoku Circuit

Each of the four data qubits is replaced by a logical qubit encoded into three physical qubits:

$$q_i \longrightarrow (q_{i1}, q_{i2}, q_{i3}), \qquad i = 1, 2, 3, 4. \tag{8.1}$$

Thus, the data register grows from 4 to 12 physical qubits. To keep the design simple, only the data qubits are encoded. Logical operations act on these encoded blocks. For example,

$$X_L = X \otimes X \otimes X, \tag{8.2}$$

and the logical CNOT is implemented transversally:

$$\mathrm{CNOT}_L = \mathrm{CNOT}_{1,1}, \mathrm{CNOT}_{2,2}, \mathrm{CNOT}_{3,3}. \tag{8.3}$$

This transversal structure prevents error propagation: a single physical error can spread to at most one corresponding qubit in the other block, ensuring that errors remain correctable. It is also convenient for implementation, since the Sudoku oracle relies heavily on XOR logic, which is naturally realized using CNOT gates. Figure 3 illustrates the structure of a protected Grover iteration.

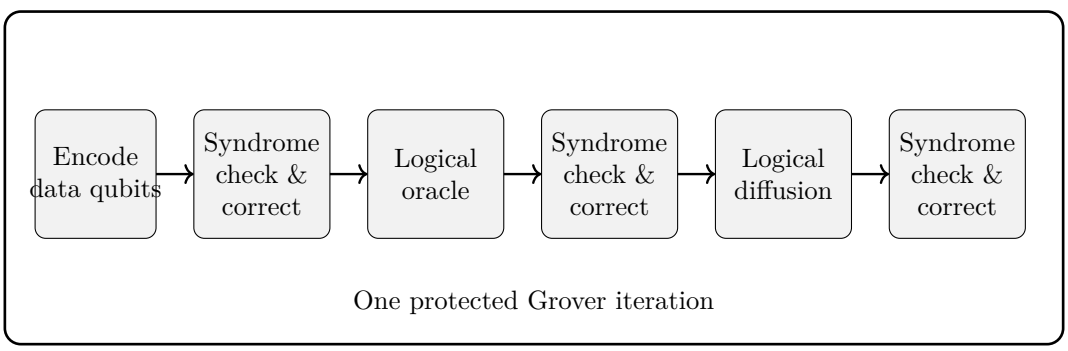


Figure 3: Circuit structure of the protected version.. Syndrome extraction and correction are inserted between major algorithmic steps.

9 Expected Results

In the ideal, noise-free setting, Grover's algorithm amplifies the marked solution. For $N = 16$, the optimal number of iterations is $T \approx \frac{\pi}{4}\sqrt{N} \approx 3$, yielding the correct state $|0110\rangle$ with high

probability P. In the presence of bit-flip noise at rate p, accumulated errors reduce the success probability to P_{uncorr}. Repetition-code protection corrects single errors, lowering the effective logical error rate from $O(p)$ to $O(p^2)$, so that in a moderate noise regime, $P_{\text{corr}} > P_{\text{uncorr}}$.

10 From Repetition Codes to Surface Codes

The three-qubit repetition code introduced earlier provides a first example of quantum error correction. However, this protection is limited because it corrects only bit-flip (X) errors and does not protect against phase errors (Z).

10.1 Repetition code as a 1D chain

The repetition code can be viewed as a one-dimensional lattice of qubits with local parity checks Z_1Z_2, Z_2Z_3. A single bit-flip error creates two violated checks (a *syndrome pair*), revealing the endpoints of the error. More generally, multiple errors form a *chain*, and only its endpoints are detected: flips occurring between the endpoints leave the parity checks unchanged.

10.2 Stabilizer formalism

The repetition code, and more generally surface codes, are instances of stabilizer codes [2, 3]. We briefly recall the formal structure.

Let $\mathcal{P}_n$ denote the n-qubit Pauli group. A stabilizer code is defined by an abelian subgroup $S \subset \mathcal{P}_n$ such that $-I \notin S$.

Definition 10.1 (Code space)**.** The code space is the simultaneous $+1$ eigenspace of all stabilizers:

$$\mathcal{C} = \{|\psi\rangle \in (\mathbb{C}^2)^{\otimes n} \ : \ g\,|\psi\rangle = |\psi\rangle \quad \forall g \in S\}.$$

Definition 10.2 (Logical operators)**.** A Pauli operator $L \in \mathcal{P}_n$ is a logical operator if it commutes with all stabilizers ($[L, S] = 0$) but is not itself in S. Logical operators act nontrivially within $\mathcal{C}$.

Remark 10.1. Error detection arises from commutation: if an error anticommutes with a stabilizer, the corresponding measurement outcome flips from $+1$ to -1. The collection of these outcomes forms the syndrome.

When no error is present, the state lies in the code space and all stabilizer measurements return $+1$, leaving the logical state unchanged. When an error occurs, the state is mapped outside the code space, and any stabilizer that anticommutes with the error returns -1. This change in measurement outcomes identifies the error without revealing or disturbing the encoded logical information.

10.3 Extending to two dimensions

Surface codes generalize this structure by arranging qubits on a two-dimensional lattice, thereby enabling protection against both bit-flip (X) and phase-flip (Z) errors [4, 5]. Qubits are placed on

edges of a square grid, and two types of local stabilizers are defined:

$$A_v = \prod_{e \ni v} X_e \quad \text{(vertex stabilizers)}, \tag{10.1}$$

$$B_p = \prod_{e \in \partial p} Z_e \quad \text{(plaquette stabilizers)}. \tag{10.2}$$

The stabilizer group consists of all products of the vertex and plaquette stabilizers:

$$S = \{\, \text{all finite products of } A_v \text{ and } B_p \,\}.$$

The corresponding code space is therefore

$$\mathcal{C}_{\text{surface}} = \{|\psi\rangle : A_v\,|\psi\rangle = |\psi\rangle\,,\ B_p\,|\psi\rangle = |\psi\rangle\ \ \forall v, p\}\,.$$

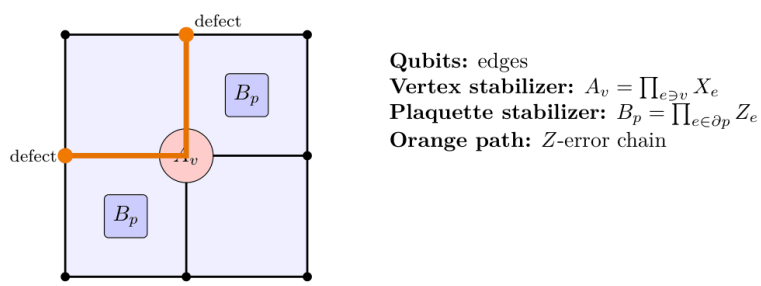


Figure 4: Surface-code lattice. Qubits are placed on edges, with local vertex (A_v) and plaquette (B_p) stabilizers. Errors form chains on the lattice, and only their endpoints—corresponding to syndrome defects—are observable.

Remark 10.2. Each stabilizer enforces a local parity constraint on the code space. Collectively, they carve out a highly constrained subspace of the full Hilbert space in which logical information is stored nonlocally.

10.4 Errors as geometric objects

The key conceptual shift in surface codes is geometric, as illustrated in Figure 4.

- Errors correspond to subsets of edges (chains), as illustrated by the red chain in Figure 4.
- Stabilizers measure local parity and detect only the boundary of these error chains.
- Distinct error chains can produce the same syndrome, since stabilizers are insensitive to the interior of a chain.

Formally, an error operator can be written as

$$E = \prod_{e \in \gamma} Z_e,$$

where γ is a set of edges. The syndrome is determined by the set of vertices at which γ has odd incidence, namely the boundary $\partial\gamma$. Error correction is therefore an inference problem: given the observed boundary $\partial\gamma$, one must infer an error chain γ consistent with the syndrome, recognizing that many distinct chains share the same boundary [2].

Closed loops play a special role: they have no boundary and are therefore undetectable by stabilizer measurements. Such loops are trivial if they can be written as products of local stabilizers and thus act as the identity, and nontrivial otherwise.

In one dimension, the analogous situation is a string of bit-flip errors spanning the entire repetition code (e.g., three flips on a three-qubit block), which produces no syndrome yet corresponds to a logical error.

Remark 10.3. Logical operators correspond to nontrivial loops since they act nontrivially on the encoded logical information.

10.5 Key insight

The transition from repetition codes to surface codes represents a shift from **redundancy** to **geometry**. This distinction is summarized in the following table:

Feature	Repetition Code (1D)	Surface Code (2D)
Error structure	One-dimensional chain	Two-dimensional lattice
Detected object	Syndrome endpoints	Syndrome defects
Error types	Bit-flip errors (X)	Bit- and phase-flip errors (X, Z)
Logical errors	System-spanning chains	Nontrivial topological paths

Remark 10.4. Surface codes are higher-dimensional stabilizer codes in which logical information is stored in global topological structure rather than local redundancy.

11 Conclusion

This report develops a complete small-scale quantum-search implementation for a 2×2 Sudoku. In the ideal setting, a reversible oracle marks the valid assignment, the diffusion operator amplifies it, and measurement yields the correct solution with high probability.

In the protected version, each data qubit is encoded using a three-qubit repetition code, with syndrome extraction interleaved to correct single bit-flip errors. More broadly, the transition from repetition codes to surface codes highlights a central principle of quantum error correction: reliability is achieved not merely through redundancy, but through geometric and topological structure.

Together, these results show that quantum algorithms and error-correcting codes must be designed jointly, and emphasize the role of topology in enabling robust quantum computation.

Appendices

A Qiskit Sketch for the Ideal Circuit

```
from qiskit import QuantumCircuit, QuantumRegister, ClassicalRegister
from qiskit_aer import AerSimulator
from qiskit import transpile

data = QuantumRegister(4, "q")
anc = QuantumRegister(5, "a")
phase = QuantumRegister(1, "p")
cl = ClassicalRegister(4, "c")

qc = QuantumCircuit(data, anc, phase, cl)

# Prepare phase ancilla in |->
qc.x(phase[0])
qc.h(phase[0])

# Put all 16 candidates into equal superposition
qc.h(data)

def sudoku_oracle(qc, data, anc, phase):
    # a1 = NOT(q1)
    qc.x(anc[0])
    qc.cx(data[0], anc[0])

    # a2 = q1 XOR q2
    qc.cx(data[0], anc[1])
    qc.cx(data[1], anc[1])

    # a3 = q3 XOR q4
    qc.cx(data[2], anc[2])
    qc.cx(data[3], anc[2])

    # a4 = q1 XOR q3
    qc.cx(data[0], anc[3])
    qc.cx(data[2], anc[3])

    # a5 = q2 XOR q4
    qc.cx(data[1], anc[4])
    qc.cx(data[3], anc[4])

    # Mark valid state by phase kickback
    qc.mcx([anc[0], anc[1], anc[2], anc[3], anc[4]], phase[0])

    # Uncompute ancillas
    qc.cx(data[3], anc[4])
    qc.cx(data[1], anc[4])
```

```
    qc.cx(data[2], anc[3])
    qc.cx(data[0], anc[3])

    qc.cx(data[3], anc[2])
    qc.cx(data[2], anc[2])

    qc.cx(data[1], anc[1])
    qc.cx(data[0], anc[1])

    qc.cx(data[0], anc[0])
    qc.x(anc[0])

def diffusion(qc, data):
    qc.h(data)
    qc.x(data)
    qc.h(data[3])
    qc.mcx([data[0], data[1], data[2]], data[3])
    qc.h(data[3])
    qc.x(data)
    qc.h(data)

for _ in range(3):
    sudoku_oracle(qc, data, anc, phase)
    diffusion(qc, data)

qc.measure(data, cl)

sim = AerSimulator()
tqc = transpile(qc, sim)
result = sim.run(tqc, shots=4096).result()
counts = result.get_counts()
print(counts)
```

B Possible Extensions

Natural next steps include:

- simulating explicit bit-flip noise,
- comparing success probabilities with and without repetition-code protection,
- plotting measurement histograms,
- or replacing the repetition code with a code that also protects phase information.

References

[1] L. K. Grover, "A fast quantum mechanical algorithm for database search," *Proceedings of the 28th Annual ACM Symposium on Theory of Computing*, 1996.

[2] M. A. Nielsen and I. L. Chuang, *Quantum Computation and Quantum Information*, Cambridge University Press, 2010.

[3] A. M. Steane, "Error correcting codes in quantum theory," *Physical Review Letters*, vol. 77, 1996.

[4] A. Y. Kitaev, "Fault-tolerant quantum computation by anyons," *Annals of Physics*, vol. 303, 2003.

[5] A. G. Fowler, M. Mariantoni, J. M. Martinis, and A. N. Cleland, "Surface codes: Towards practical large-scale quantum computation," *Physical Review A*, vol. 86, 2012.

[6] P. W. Shor, "Scheme for reducing decoherence in quantum computer memory," *Physical Review A*, vol. 52, no. 4, 1995.